



Giới thiệu Generics trong Java

NỘI DUNG

1. Generic là gì?

2. Giải pháp: Generic

3. Generic trong Collection

4. Vấn đề: Generic quá chung

5. Bounded Generic và Wildcard

6. Lợi ích của Generic

7. Tổng kết

1. Generic là gì?

Generic = Kiểu dữ liệu tổng quát

Cho phép tham số hóa kiểu dữ liệu

Áp dụng cho: Class, Method, Interface

Generic có thể hiểu đơn giản là cho phép chúng ta **viết code mà chưa cần xác định kiểu dữ liệu cụ thể**. Kiểu dữ liệu sẽ được quyết định khi **sử dụng**, không phải khi **khai báo**. Generics thường xuất hiện trong Collection như `ArrayList<String>`, `List<Integer>`...

Vấn đề khi không dùng Generic

```
ArrayList list = new ArrayList();
```

```
list.add("Hello");
```

```
list.add(10);
```

```
String s = (String) list.get(1); // Lỗi runtime
```

Trước khi có Generics, Java dùng Object. Điều này dẫn đến **phải ép kiểu** và lỗi chỉ phát hiện khi **chạy chương trình** (runtime error). Đây là điều rất nguy hiểm với các hệ thống lớn.

2. Giải pháp: Generic

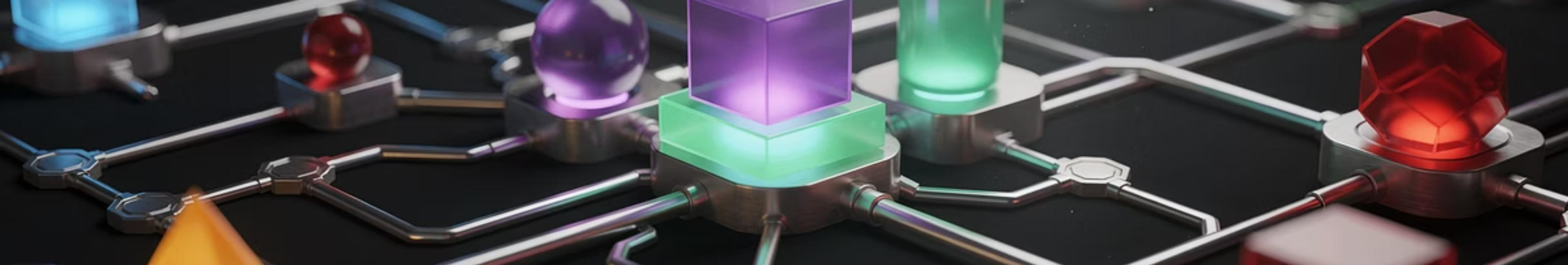
```
ArrayList<String> list = new ArrayList<>();  
  
list.add("Hello");  
  
// list.add(10);
```

Generic Class

```
class Box<T> {  
  
    private T value;  
  
    public void setValue(T value) {  
  
        this.value = value;  
  
    }  
  
    public T getValue() {  
  
        return value;  
  
    }  
  
}
```

Đây là một Generic Class. **T** là một **tham số kiểu dữ liệu**, có thể thay bằng bất kỳ kiểu nào khi tạo đối tượng.

Tên thường dùng: T (Type), E (Element), K (Key), V (Value).



Sử dụng Generic Class

```
Box<String> box1 = new Box<>();  
  
box1.setValue("Hello");  
  
Box<Integer> box2 = new Box<>();  
  
box2.setValue(100);
```

Cùng một class Box nhưng có thể dùng cho nhiều kiểu dữ liệu khác nhau. Đây chính là **tái sử dụng code** – một ưu điểm rất lớn của Generic.

Generic Method

```
public static <T> void printArray(T[] arr) {  
  
    for (T item : arr) {  
  
        System.out.println(item);  
  
    }  
  
}
```

Ngoài class, chúng ta còn có **Generic Method**. Kiểu dữ liệu được khai báo **trước kiểu trả về**. Một method nhưng có thể xử lý nhiều kiểu dữ liệu khác nhau.



The diagram consists of three rounded rectangular boxes with red borders. The first box on the left contains the text **List<String>**. The second box on the right contains the text **Set<Integer>**. The third box, positioned below the first two, contains the text **Map<String, Integer>**.

Generics được dùng rất nhiều trong Collection Framework.
Điều này giúp hạn chế lỗi, tăng khả năng đọc hiểu và hỗ trợ IDE tốt hơn.

4. Vấn đề: Generic quá chung

- ☐ T quá tổng quát
- ☐ Không gọi được method riêng

```
T value;
```

```
value.compareTo(...); //
```

5. Bounded Generic và Wildcard

```
class Calculator<T extends Number> {  
  
    public double doubleValue(T value) {  
  
        return value.doubleValue();  
  
    }  
  
}
```

Giải thích

- T extends Number
- Chỉ chấp nhận Integer, Double, Float, ...

Bounded Generic giúp giới hạn kiểu dữ liệu đảm bảo T có các method cần thiết.

Giới thiệu Wildcard (?)

```
List<?> list;
```



? = không xác định kiểu cụ thể



Dừng khi chỉ đọc dữ liệu

Wildcard cho phép ta làm việc với **nhiều kiểu Generic khác nhau** mà không quan tâm kiểu cụ thể là gì.

Các loại Wildcard

01

Unbounded Wildcard

```
List<?> list;
```

02

Upper Bounded Wildcard

```
List<? extends Number>
```

03

Lower Bounded Wildcard

```
List<? super Integer>
```

- extends → **chỉ đọc**
- super → **chỉ ghi**
- **!** Rule nổi tiếng: **PECS**
 - *Producer Extends*
 - **Consumer Super**

So sánh: Generic vs Wildcard

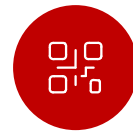
Generic	Wildcard
Định nghĩa kiểu	Không định nghĩa kiểu
Dùng khi tạo class	Dùng khi truyền tham số
Có thể set & get	Thường chỉ get

Generic dùng để **thiết kế**, Wildcard dùng để **sử dụng linh hoạt**.

6. Lợi ích của Generic



An toàn kiểu dữ liệu (Type Safety)



Không cần ép kiểu



Tái sử dụng code



Dễ bảo trì

Tóm lại, Generics giúp code **an toàn hơn, ngắn gọn hơn và chuyên nghiệp hơn** – đặc biệt trong các dự án lớn.

7. Tổng kết

- ☐ Generic giúp code an toàn & linh hoạt
- ☐ Bounded Generic giới hạn kiểu
- ☐ Wildcard tăng tính tái sử dụng
- ☐ Rất quan trọng trong Collection Framework

Nắm vững Generic, Wildcard và Bounded Generic sẽ giúp bạn đọc hiểu tốt code Java nâng cao, đặc biệt là Collection & Framework.